

Claude Code CLI Commands

Complete Reference — Real vs Invented

Every real flag · Every in-session command · All invented distractors · Scenario → command table · The 3-way distinction

Real CLI Flags — The Complete List

Every flag that actually exists in Claude Code

The golden rule: if you don't recognise a flag, it almost certainly doesn't exist. Memorise this list so invented distractors feel wrong immediately.

--name

Starts a named session that persists to storage as it runs

Use: Any work you want to resume later after closing or interruption

Exam signal: 'named session', 'persist', 'save for later', 'plan to resume'

--resume

Rehydrates a previously named session — restores full context and conversation history

Use: Reconnecting after disconnect, SSH drop, or continuing interrupted long-running work

Exam signal: 'resume', 'reconnect', 'restore', 'continue', '20 turns of context', 'SSH drops'

-p / --print

Non-interactive mode — processes the prompt and exits immediately

Use: CI/CD pipelines, automation scripts, any non-interactive automated use

Exam signal: 'non-interactive', 'pipeline', 'CI', 'automated', 'hangs waiting for input'

**--output-format
json**

Returns structured JSON output instead of rendered markdown

Use: Programmatic parsing of Claude's output by downstream pipeline steps

Exam signal: 'parse output', 'downstream system', 'structured', 'machine-readable'

**--output-format
stream-json**

Streams JSON output in real-time as it generates

Use: Real-time processing of long outputs in live pipelines

Exam signal: 'streaming', 'real-time pipeline', 'progressive output'

--model

Specifies which Claude model to use for this session

Use: Overriding the default model — e.g. claude-opus-4-6 for complex tasks

Exam signal: 'specific model', 'use Opus', 'use Haiku', 'model selection'

--allowedTools

Restricts which tools this session is permitted to use

Use: Security scoping and least-privilege enforcement for a specific run

Exam signal: 'restrict tools', 'limit capabilities', 'least privilege', 'only allow'

--system-prompt

Injects a system prompt from the CLI, overriding any CLAUDE.md for this run

Use: Overriding default instructions for a specific one-off invocation

Exam signal: 'override instructions', 'custom prompt', 'ignore CLAUDE.md for this run'

- no - tools

Disables all tools — forces a pure text reasoning response

Use: When you want reasoning or analysis only, with no file system access

Exam signal: 'text only', 'no file access', 'pure reasoning', 'disable tools'

fork_session

Branches from a named session checkpoint — creates an independent copy

Use: Exploring two different approaches from the same starting point without affecting the original

Exam signal: 'branch', 'explore two paths', 'divergent', 'from checkpoint', 'independent copy'

Real In-Session Slash Commands			
Commands typed during an active Claude Code session			
Command	What it does	Use case	Exam signal words
/memory	Shows exactly which CLAUDE.md and rules files are currently loaded for this session	Debug why Claude is ignoring instructions or behaving inconsistently	'debug why Claude ignores instructions', 'check what files are loaded', 'instructions not followed'
/compact	Compresses conversation history to free context window space while preserving ability to continue	Long sessions where the context window is filling up	'context window filling up', 'long session', 'running out of context', 'compress history'
/clear	Clears conversation history entirely and starts fresh in the same session	Starting over without closing the session	'start over', 'reset context', 'fresh start', 'clear everything'
/status	Shows current session status — model, tools, and configuration	Checking current config mid-session	'check current config', 'what model am I using', 'session info'
/help	Lists all available built-in and custom commands	Discovering available commands	—
/	Runs a custom command defined in .claude/commands/ (project-scoped) or ~/.claude/commands/ (user-scoped)	Invoking a saved workflow — e.g. /review-pr, /generate-tests	'run the review workflow', 'trigger a slash command', 'invoke a skill'

Real vs Invented — Eliminate Instantly

The exam invents plausible-sounding flags that don't exist

■ INVENTED — eliminate immediately • --recover last ← exam question mistake • --recover (any form) • --restore • --reconnect • --continue • --reload • --checkpoint • --replay • --sync • --persist • --history • .claude_history (file doesn't exist) • 'memory buffer' (not a real concept) • --rollback • --reattach	■ REAL — only these exist • --name start named persistent session • --resume rehydrate after disconnect • -p / --print non-interactive CI mode • --output-format json machine-parseable • --output-format stream-json streaming • --model select model • --allowedTools restrict tools • --system-prompt override prompt • --no-tools text-only mode • fork_session branch from checkpoint • /memory debug loaded files • /compact compress conversation • /clear reset context • /status session info • / custom slash command
--	--

KEY

The exam uses real developer vocabulary to invent fake flags: --recover, --restore, --reconnect all feel like they should exist. They don't. The ENTIRE 'resume/reconnect/continue' use case is handled by exactly ONE flag: --resume .

The 3-Way Distinction — Most Tested CLI Question Type

--name vs --resume vs fork_session

This is the exam's favourite CLI question. All three involve persisted sessions but serve completely different purposes.

Command	When to use	What it does	CI/CD safe?	Key distinction
--name	STARTING a session you plan to resume later	Creates a named session and persists its state to storage as the session runs	No — interactive use only	This is the BEGINNING. Use at session START, not when reconnecting.
--resume	RECONNECTING to continue the SAME session after interruption	Loads the full persisted state — restores all turns, context, and history. Same thread, continued.	Never in CI — breaks isolation between runs	This is CONTINUATION. Only works if --name was used when the session started.
fork_session	Branching FROM a checkpoint to go in a DIFFERENT direction	Creates an independent copy from the saved state. Original session unaffected. Two divergent paths.	No — interactive use only	This is BRANCHING. Use when you want to explore two approaches from the same starting point.
-p (no --resume)	CI/CD automated runs — fresh every time	Non-interactive, completely isolated, reproducible. No session carryover from previous runs.	Yes — always use in CI	This is ISOLATION. Every CI run starts clean. Never combine with --resume in pipelines.

EXAM TRAP — --RESUME IN CI	Using --resume in CI/CD pipeline jobs to 'carry over context from the previous run.' Wrong — CI jobs must be isolated and reproducible. Use -p without --resume so each run evaluates the current state independently.
EXAM TRAP — FORK_SESSION FOR RECONNECTION	Using fork_session after an SSH drop to 'restore the session from the checkpoint.' Wrong — fork_session creates a BRANCH (independent copy). For reconnection, use --resume to continue the same thread.
EXAM TRAP — --NAME FOR RESUMING	Using --name when reconnecting after a disconnect. Wrong — --name creates a NEW named session (or overwrites). For resuming, use --resume . The session must have been started with --name originally.

Scenario → Correct Command — Know These Cold

Every CLI exam scenario mapped to its answer

Exam scenario	Correct command / action	Wrong options to eliminate
SSH drops mid-session. Need to continue with all 20 turns of context.	--resume (rehydrates full state)	--recover / --reconnect / fork_session / new session / any invented flag
Starting a long debugging session you plan to return to tomorrow.	--name (creates the persistent named session)	--resume (for resuming, not starting) / -p (non-interactive) / fork_session
You've analysed a codebase. Want to explore Fix A and Fix B independently from the same baseline.	fork_session (creates independent branches)	--resume (continues same thread, no branching) / two new sessions (no shared baseline)
CI/CD pipeline for automated PR review. Must not hang.	-p --output-format json (non-interactive + structured output)	--resume (breaks isolation) / --name (interactive) / plan mode (requires approval)
Context window filling up mid-session. Don't want to lose the thread.	/compact (in-session command — compresses history)	New session (loses all context) / --resume (doesn't reduce window size)
Claude Code is ignoring CLAUDE.md instructions. Need to debug.	/memory (shows which files are currently loaded for this session)	Rewriting CLAUDE.md without checking if it loads / --resume / new session
Need to run Claude non-interactively and parse its output in a script.	-p --output-format json	--output-format stream-json (streaming, harder to parse) / prompt-based JSON (unreliable)
Need to use a specific model (Opus) for a complex analysis task.	--model claude-opus-4-6	--system-prompt (controls instructions, not model) / --allowedTools (controls tools)
Want to restrict Claude to only reading files, no Bash execution.	--allowedTools Read,Write,Edit (excludes Bash)	--no-tools (disables everything) / system prompt instructions (not enforced)
Need a pure reasoning response with no tool calls at all.	--no-tools (disables all tools — text only)	tool_choice: "none" (API-level, not CLI) / system prompt "don't use tools" (probabilistic)

Command Pair Relationships — What Goes Together

These commands only work in pairs. Understanding the relationship prevents the most common CLI mistakes.

--name + --resume

--name at START

Creates the persistent session. Must be used first or --resume has nothing to load.

--resume to CONTINUE

Only works if the session was originally started with --name. Rehydrates full state.

-p + --output-format json

-p (non-interactive mode)

Makes Claude Code exit after processing rather than wait for input. MANDATORY for CI.

--output-format json (structured output)

Makes the output machine-parseable. Always pair with -p for CI pipeline use.

fork_session + --name	
--name (create the base) Create the named session first. This becomes the shared baseline for both branches.	fork_session (create branches) Branch from the named session. Each fork is independent — changes don't affect each other or the original.

Quick-Fire — Exam Signal →
Correct Answer

Read the signal word, pick the flag in under 3 seconds

Signal word / phrase in question	Answer
'SSH drops', 'disconnect', 'reconnect', 'restore context', 'N turns of history'	--resume
'start a session', 'named session', 'plan to resume later', 'persist'	--name
'two approaches', 'branch', 'diverge', 'explore independently', 'from checkpoint'	fork_session
'CI/CD', 'pipeline', 'non-interactive', 'automated', 'hangs'	-p (mandatory)
'parse output', 'downstream script', 'machine-readable', 'structured'	--output-format json
'context window filling', 'long session', 'running out of space'	/compact (in-session)
'ignoring CLAUDE.md', 'instructions not followed', 'debug what loads'	/memory (in-session)
'start over', 'reset', 'fresh context'	/clear (in-session)
'restrict tools', 'least privilege', 'only allow Read'	--allowedTools
'no tools', 'text only', 'pure reasoning'	--no-tools
'specific model', 'use Opus', 'use Haiku'	--model
'override instructions', 'one-off system prompt'	--system-prompt
Any flag you don't recognise in an option	Eliminate that option — it's invented

Claude Code CLI Commands reference complete. Pairs with: CCA-F_Layers_Fix_At_Right_Layer.pdf and CCA-F_Anthropic_Guidance_Principles.pdf